



CMSC 180

Introduction to Parallel Computing Second Semester AY 2025-2026

Laboratory Research Problem 03 (LRP03)

Core-affine Threaded Min-Max Transformation of Elements in a Column of a Matrix

Introduction

Rewrite the program that you wrote in Laboratory Research Problem 02 (LRP02) so that the threads that you created each must be ran specifically unto one specific core only. If your computing machine has four or eight threads, assign your threads to three or seven cores, respectively.

Research Activity 1: How to do it?

1. Write the main program `lab03` (just rewrite `lab02`) that includes the following:
 - (1) Read n and t as user inputs (maybe from a command line or as a data stream), where n is the size of the square matrix, t is the number of threads to create, and $n \gg t$;
 - (2) Create a non-zero $n \times n$ square matrix $\mathbf{X}$ whose elements are assigned with random integer;
 - (3) Divide your $\mathbf{X}$ into t submatrices of size $n \times n/t$ each, $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$;
 - (4) Take note of the system time `time_before`;
 - (5) Create t threads, where for the i th thread call `mmt( $\mathbf{x}_i, n, n/t$ )`; ← **very important**
 - (6) Assign the thread to a core; ← **very important, this is the additional step in this exercise**
 - (7) Recreate the correct $\mathbf{T}$ from the output of each threaded `mmt()`; ← **very important**
 - (8) Take note of the system time `time_after`;
 - (9) Obtain the elapsed time `time_elapsed := time_after - time_before`;
 - (10) Output `time_elapsed`;
2. Fill in the following table:

n	t	Time Elapsed (seconds)			Average Runtime (seconds)
		Run 1	Run 2	Run 3	
25,000	1				
25,000	2				
25,000	4				
25,000	8				
25,000	16				
25,000	32				
25,000	64				

Research Question 1: What is the difference of the average time that you obtained for $t = 1$ compared to the average that was obtained in LRP01 and LRP02?

Research Activity 2: Repeat research activity 1 for $n = 30,000$ and $n = 40,000$. Do you think you can now achieve $n = 50,000$ and even $n = 100,000$? Try it to see if you can. If you were able to do so, why do you think you can now do it? If not yet, why do you think you still can not?

3. Using a graphing software for each n , graph t versus **Average** obtained from the Table above. Label the line as “With core affinity.” Then superimpose the graph that you obtained in LRP02 and label that line as “Without core affinity.” Superimpose also the graph that you obtained in LRP01 and label it as “Serial.” Describe in detail what you have observed. Do you think you can go as far as $t = n$? If not, what about $t = n/2$? Or, $t = n/4$? Or, $t = n/8$?

Research Question 2: Did your average time improve compared to LRP01 and LRP02 for every t ? Which t is the average time better? Statistically the same? Slower? Why?

Research Question 3: Discuss in your own words what will happen if instead you divide **X** as in step (3) above, still into t submatrices, but in a manner where each submatrix is of size $n/t \times n$?

Solutions to Possible Problems that may Come up

Problem 1 : *I do not know how to write core-affine threaded programs.*

Answer 1 : *Is that really a problem?*

Problem 2 : *The programming language I used for LRP01 and LRP02 does not have a robust library for writing core-affine threaded codes.*

Answer 2 : *See Answer 1.*

Some Helpful Online Resources (also listed in Google Classroom)

1. C/C++ Example from bytefreaks.net
2. C++ Threads, Affinity, and Hyperthreading from Dzone
3. Python `setaffinity()` method from geeksforgeeks.org
4. Perl's `CpuAffinity()` from metacpan.org